

Тестирование идиempотентности и повторяемости запросов в сетевых API

по дисциплине «Методы тестирования программного обеспечения»

Плахотников В. А.

СПбПУ, ИКНТ ВШ ПИ

2026

Выполнил студент группы 5130903/30303:
Плахотников В. А.

Руководитель:
Старший преподаватель В. А. Пархоменко

Структура доклада

- 1 Постановка проблемы
- 2 Математическая формализация
- 3 Алгоритм
- 4 Шесть типов контроллеров
- 5 Иллюстративный пример
- 6 Апробация
- 7 Выводы

Проблема: При сетевых сбоях клиент повторяет HTTP-запрос (retry).

Без идемпотентности:

- POST-запрос на создание заказа $\times 3$ retry $\Rightarrow$ 3 заказа
- POST-запрос на платёж $\times 3$ retry $\Rightarrow$ **тройное списание**
- POST товара с тем же SKU $\Rightarrow$ дубликат в каталоге

Цель: Исследовать и продемонстрировать тестирование идемпотентности сетевых API на примере шести различных типов контроллеров Spring Boot (REST, SOAP, gRPC, GraphQL).

Определение. Операция $O : S \rightarrow S$ *идемпотентна*, если:

$$O^n(s) = O(s), \quad \forall n \geq 1, \forall s \in S$$

HTTP-методы (RFC 9110):

Метод	Safe	Идемпотентный
GET, HEAD	✓	✓
PUT, DELETE	—	✓
POST, PATCH	—	—

Решение для POST: Idempotency-Key k :

$$O'(s, r, k) = \begin{cases} O(s, r), & k \notin \text{KeyStore} \\ \text{cached}(k), & k \in \text{KeyStore} \end{cases}$$

Алгоритм 1 CheckIdempotency(*request*)

```
1: если request.method  $\neq$  POST тогда
2:   вернуть ProcessRequest(request)
3: конец если
4: key  $\leftarrow$  headers["Idempotency-Key"]
5: если key =  $\emptyset$  тогда
6:   вернуть 400 Bad Request
7: конец если
8: cached  $\leftarrow$  KeyStore.find(key)
9: если cached  $\neq \emptyset$  и cached.response  $\neq \emptyset$  тогда
10:  вернуть cached.response
11: конец если
12: KeyStore.register(key)
13: response  $\leftarrow$  ProcessRequest(request)
14: KeyStore.save(key, response)
15: вернуть response
```

▷ кэшированный ответ

Шесть кардинально разных типов контроллеров

Тип	Транспорт	Домен	Идемпотентность
@RestController	HTTP/REST (JSON)	Заказы	Idempotency-Key (HTTP-фильтр)
Functional Endpoints	HTTP/REST (JSON)	Платежи	тот же фильтр
Spring Data REST	HTTP/HAL+JSON	Товары	@Version + ETag
Spring-WS @Endpoint	SOAP / HTTP+XML	Отправления	тот же фильтр
GraphQL @Controller	GraphQL / HTTP	Инвентарь	тот же фильтр
@GrpcService	gRPC / HTTP/2	Уведомления	ServerInterceptor

Ключевая идея: четыре из шести транспортов используют *один и тот же* IdempotencyFilter; gRPC — единственный, требующий отдельного перехватчика, но он делегирует тому же IdempotencyService.

Иллюстративный пример: retry платежа

Сценарий: Клиент отправляет POST /api/payments, получает таймаут, повторяет запрос.

С идемпотентностью (Idempotency-Key: "pay-001"):

- 1 Запрос #1: key="pay-001" $\notin$ KeyStore $\Rightarrow$ register, создать платёж, сохранить ответ
- 2 Запрос #2 (retry): key="pay-001" $\in$ KeyStore $\Rightarrow$ **вернуть кэш**
- 3 Результат: **1 платёж** (корректно)

Без идемпотентности:

- 1 Запрос #1: создать платёж $\Rightarrow$ списание #1
- 2 Запрос #2 (retry): создать платёж $\Rightarrow$ списание #2
- 3 Результат: **2 платежа — двойное списание!**

Пошаговое выполнение на данных

Вход: POST /api/orders, body: {description: "Ноутбук", amount: 89999.99}

Шаг	Действие	KeyStore	orders (count)
0	Начальное состояние	∅	0
1	Получен key="abc"	∅	0
2	find("abc") = ∅	∅	0
3	register("abc")	{abc: pending}	0
4	INSERT order	{abc: pending}	1
5	save("abc", 201, body)	{abc: 201, body}	1
6	Retry: key="abc"	{abc: 201, body}	1
7	find("abc") ≠ ∅	{abc: 201, body}	1
8	return cached 201	{abc: 201, body}	1

Результат: 1 заказ, 2-й запрос вернул кэш. Идемпотентность обеспечена.

Технологии: JUnit 5 + Testcontainers (PostgreSQL в Docker) + MockMvc

Тестовый класс	Тестов	Транспорт	Результат
OrderIdempotencyTest	6	@RestController	PASS
PaymentIdempotencyTest	5	Functional	PASS
ProductIdempotencyTest	5	Spring Data REST	PASS
SoapIdempotencyTest	3	SOAP (Spring-WS)	PASS
GrpcIdempotencyTest	4	gRPC (HTTP/2 + protobuf)	PASS
GraphQLIdempotencyTest	4	GraphQL	PASS
DataImportTest	2	JSON/CSV	PASS
NonIdempotentProfileTest	7	все 6 транспортов	PASS*
Итого	36		PASS

*NonIdempotentProfileTest: 7 тестов (по одному на каждый транспорт + ранее существующие), все проходят зелёными, **подтверждая наличие багов** при отключённой идемпотентности.

Профиль non-idempotent: демонстрация проблем на 6 транспортах

Тест	Проблема	Статус
REST POST без Idempotency-Key	Проходит без проверки	BUG
REST повтор POST с тем же ключом	Дубликат заказа	BUG
Data REST дубликат SKU	Проходит (нет constraint)	BUG
Functional 3x retry платежа	Тройное списание	BUG
SOAP тот же tracking_number	Два Shipment с одним номером	BUG
gRPC тот же idempotency-key	Interceptor выкл. ⇒ дубликат	BUG
GraphQL мутация с тем же ключом	Два Inventory с одним item_name	BUG

Ключевой вывод: отключение механизмов идемпотентности приводит к критическим багам *во всех транспортах*, и каждый случай обнаруживается отдельным тестом.

Единый `IdempotencyService` обслуживает шесть транспортов через два механизма перехвата.

Перехват	Транспорты	Кодировка снимшота
<code>IdempotencyFilter (servlet)</code>	REST, Functional, SOAP, GraphQL	TEXT (JSON / XML)
<code>GrpcIdempotencyInterceptor</code>	gRPC	BYTEA (protobuf)
<code>@Version + ETag</code>	Spring Data REST	—

Доработка фильтра: миграция V4 расширила `idempotency_keys` тремя колонками — `response_content_type`, `response_kind`, `response_body_bytes`. Это превратило фильтр из «JSON-only» в *транспорт-агностичный*: SOAP реплеит `text/xml`, gRPC — бинарные `protobuf`-байты через `MethodDescriptor.responseMarshaller`.

- 1 Формализована математическая модель идемпотентности HTTP: $O^n(s) = O(s)$
- 2 Реализованы **6 типов контроллеров** в 4 транспортах с единым механизмом идемпотентности:
 - @RestController, Functional, Spring Data REST (HTTP/REST)
 - Spring-WS @Endpoint (SOAP)
 - @GrpcService + ServerInterceptor (gRPC HTTP/2)
 - GraphQL @Controller (single endpoint)
- 3 Доработан IdempotencyFilter до транспорт-агностичного: снимок ответа $\langle \text{status}, \text{contentType}, \text{kind}, \text{body} \rangle$ позволяет реплеить JSON, XML и бинарный protobuf через единый кеш.
- 4 **36 интеграционных тестов** (Testcontainers + PostgreSQL)
- 5 Профиль non-idempotent демонстрирует баги во всех 6 транспортах

Спасибо за внимание!